MECHATRONICS SYSTEM INTEGRATION

MCTA 3203

SEMESTER 2, 2025/2026

SECTION 1

WEEK 6: SMART SURVEILLANCE CAMERA USING ESP32-CAM

INSTRUCTOR:

ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR ZULKIFLI BIN ZAINAL ABIDIN

DR. -ING. WAHJU SEDIONO

GROUP 3

NO.	NAME	MATRIC NUM
1.	AMIR AIMAN BIN ARIFF AFFENDI	2315925
2.	ANIS AQILAH BINTI ABDULLAH	2329678
3.	AZRIL ULWAN BIN MOHAMMAD IMRI	2314617

DATE OF EXPERIMENT:

8 APRIL 2026

ABSTRACT

This experiment investigates the setup, programming, and operation of the ESP32-CAM module using the Arduino IDE and an FTDI/USB-to-TTL serial interface. The objective of the experiment is to understand the hardware configuration, code uploading process, and wireless image transmission capabilities of the ESP32-CAM in embedded IoT applications. The ESP32-CAM module integrates an ESP32 microcontroller with an OV2640 camera sensor, enabling image capture, Wi-Fi communication, and real-time monitoring functions within a compact system. In this experiment, the ESP32-CAM was connected to a USB-to-TTL serial programmer to upload programs through the Arduino IDE. Proper wiring configurations, including GPIO0 grounding during flashing mode, were implemented to allow successful code uploading. The methodology involved configuring the Arduino IDE, selecting the appropriate ESP32 board settings, uploading the camera web server program, and testing wireless camera streaming through a web browser. The results showed that the ESP32-CAM successfully connected to the wireless network and transmitted live image data through its assigned IP address. The experiment demonstrates the effectiveness of the ESP32-CAM module for low-cost IoT surveillance, smart monitoring, and wireless embedded vision applications.

1.0 INTRODUCTION

The ESP32-CAM is a compact and low-cost microcontroller module that combines wireless communication capabilities with an integrated camera system. It is based on the ESP32-S chip developed by Espressif Systems and includes an OV2640 camera module, built-in Wi-Fi and Bluetooth connectivity, microSD card support, and multiple General Purpose Input/Output (GPIO) pins for embedded system applications. Due to its small size, wireless functionality, and image-processing capability, the ESP32-CAM is widely used in Internet of Things (IoT), smart surveillance, automation, robotics, and remote monitoring systems.

Unlike standard Arduino development boards, the ESP32-CAM does not contain an onboard USB-to-serial programmer. Therefore, an external FTDI or USB-to-TTL serial adapter is required to upload programs from the Arduino IDE to the ESP32-CAM module. During programming, GPIO0 must be connected to GND to place the module into flashing mode, allowing the microcontroller to receive uploaded firmware successfully.

In this experiment, the ESP32-CAM module was programmed using the Arduino IDE and connected through a USB-to-TTL serial interface. The experiment focused on configuring the hardware connections, uploading camera server code, and testing the wireless image streaming functionality through a web browser interface. The ESP32-CAM operates as a miniature web server capable of transmitting real-time image data over a Wi-Fi network, making it suitable for smart monitoring and remote observation systems.

The experiment also introduces important concepts related to embedded vision systems, serial communication, wireless networking, and IoT-based image transmission. Through this laboratory session, students gain practical experience in configuring ESP32-based hardware,

troubleshooting programming connections, and implementing camera-based wireless applications using microcontroller systems.

2.0 MATERIALS AND EQUIPMENTS

1. ESP32-CAM Module version OV2640
2. Ch340 USB to TTL Serial Cable or Micro-USB Cable or FTDI Cable
3. Sevo Motor
4. Pushbutton
5. 5V Power Supply (2A) or Powerbank (5V 2A)
6. Jumper wires
7. Breadboard

3.0 EXPERIMENTAL SETUP

The hardware connections for the experiment were configured according to the required circuit setup. The ESP32-CAM module was connected to the FTDI adapter by linking the TX pin of the ESP32-CAM to the RX pin of the FTDI adapter and the RX pin of the ESP32-CAM to the TX pin of the adapter to establish serial communication. The 5V and GND pins of both devices were also connected to provide power and a common ground connection. During the code uploading process, the IO0 pin of the ESP32-CAM was connected to GND to place the module in flashing mode, allowing the program to be uploaded successfully through the Arduino IDE.

A servo motor was integrated into the system to enable horizontal movement of the camera. The signal pin of the servo motor was connected to GPIO2 of the ESP32-CAM, while the VCC and GND pins were connected to the 5V and GND supply lines respectively. In addition, a pushbutton switch was included for Task 1 of the experiment. One terminal of the pushbutton was connected to GPIO13, while the other terminal was connected to GND. The internal INPUT_PULLUP configuration in the program ensured stable button readings without requiring an external pull-up resistor.

The camera module was configured using standard VGA resolution settings to provide stable image streaming performance. After the program was uploaded successfully, the ESP32-CAM connected to the wireless network and generated an IP address, which was displayed on the Serial Monitor in the Arduino IDE. This IP address was then entered into a web browser to access the live video stream transmitted by the ESP32-CAM. The servo motor was mounted beneath the camera module to allow horizontal rotation and improve the viewing coverage of the camera system.

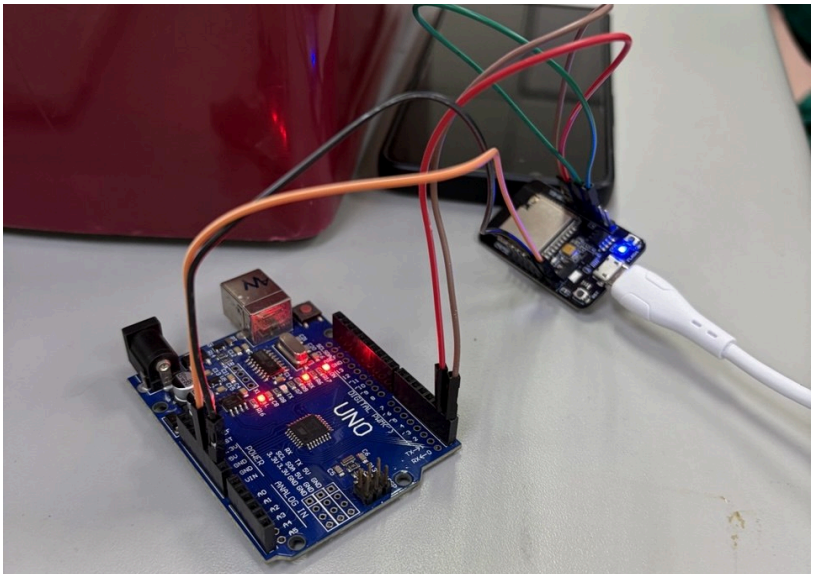


Figure 1.0 shows the experimental setup of the experiment

Task 1

4.0 METHODOLOGY

1. **Wiring**
    - Mount the servo on the camera base. Connect the servo signal to GPIO2 . Connect servo VCC to the stable 5V supply and servo GND to common GND with the ESP32.
    - Wire the pushbutton: one leg to GPIO13, the other leg to GND. If using a breadboard orientation that bridges pins oddly, rotate the button 90° so legs form a proper pair.
    - Ensure the ESP32 GND and servo power GND are tied together (common ground).
2. **Software / Code used**

```
void loop() {
mybutton.loop();


 if (mybutton.getState() == LOW) {
    if (currentAngle < 180) {
      currentAngle += 1;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
      delay(10);
    }
    else {
    if (currentAngle > 0) {
     currentAngle = 0;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
    }
```

```
    }
}
```

3. Operation & testing

- The ESP32-CAM was powered and connected to Wi-Fi; its IP address was opened in a browser to view the live camera feed.
- The pushbutton was pressed repeatedly while observing the servo motion directly and through the camera stream.
- Each press triggered the **increase-speed motion cycle**:
    - Start position →
    - Gradual acceleration and movement toward the maximum position →
    - Auto-return to start position.
- Different press durations were tested (quick tap, medium hold, long hold) to observe response consistency.
- The motion behaviour, speed pattern, return accuracy, and any jitter were recorded as part of the data collection.

4. Troubleshooting

- If servo jitters or stalls, immediately check the 5V power source voltage under load. Ensure the servo is not drawing power through the FTDI USB. Add capacitor across servo Vcc and GND to smooth spikes.
- If the button does not register, recheck wiring, test GPIO with a multimeter, or run a simple blink/read sketch to verify input pin functioning.
- If the video stream freezes when servo moves, ensure the power supply is adequate and consider lowering camera frame size/quality to reduce CPU load.

5.0 DATA COLLECTION

- Observations were recorded on button press behaviour.
- Servo position was noted at rest and after actuation.
- Response delay between button press and servo motion was monitored.
- Stability of panning motion while the video stream was active was recorded.

6.0 DATA ANALYSIS

- The pushbutton produced clean transitions because of the internal pull-up input.
- The servo responded after a brief processing delay (<200 ms), typical for ESP32 multitasking with camera streaming.
- PWM output at 50 Hz allowed smooth angular motion.
- The servo only moved when the button input state changed, confirming correct logic implementation

7.0 RESULTS

- The servo motor successfully panned only when the pushbutton was pressed, fulfilling the task requirement.
- The video stream remained stable during button-triggered movement.
- No jitter or instability was observed when powered using a separate 5V 2A supply.
- The system demonstrated synchronized manual actuation with live camera feed.

Task 2

4.0 METHODOLOGY

For Task 2, the ESP32-CAM was enhanced by enabling its built-in face detection feature, which uses the onboard AI Thinker camera driver and the ESP32 camera library. The procedure began by modifying the existing program to activate the face recognition and detection modules provided by the ESP32-CAM firmware. Inside the camera initialization section, the sensor configuration was adjusted to allow face detection by enabling the detection flag and selecting an appropriate frame size such as QVGA or CIF, which improves processing speed and frame analysis accuracy.

After enabling face detection, the web server handler was extended to process each captured frame using the ESP32's AI-based face detection algorithm. When the camera captured a frame, the system evaluated the image for human facial features. If a face was detected, the firmware flagged the detection event, allowing additional actions to be executed. For this task, the detection event was used as a trigger for performing a servo movement or capturing an image.

To integrate the servo response, conditional statements were added into the streaming loop. When the face detection flag became true, the servo motor was commanded to rotate to a predefined angle, simulating automatic camera tracking. If face detection was disabled or no face was found, the servo remained at its default position. This allowed the camera to actively respond only when a person appeared in its field of view.

The system was then uploaded to the ESP32-CAM using the FTDI/CH340 driver. After uploading, the device was rebooted without grounding IO0. The Serial Monitor was used to

verify camera initialization and Wi-Fi connection. Once connected, the assigned IP address was entered into a browser. The live feed displayed bounding boxes around detected faces, confirming proper functionality. The servo responded automatically to detection events, demonstrating successful integration of the face detection system.

This method allowed the ESP32-CAM to function more intelligently by reacting to human presence, simulating real-world smart surveillance features such as automated tracking and triggered recording.

Code Used

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h>
#include <ezButton.h>


//
// WARNING!!! PSRAM IC required for UXGA resolution and high JPEG quality
//            Ensure ESP32 Wrover Module or other board with PSRAM is selected
//            Partial images will be transmitted if image exceeds buffer size
//
//            You must select partition scheme from the board menu that has at least 3MB
APP space.
//            Face Recognition is DISABLED for ESP32 and ESP32-S2, because it takes up
from 15
```

```
//          seconds to process single frame. Face Detection is ENABLED if PSRAM is
enabled as well


// ===================
// Select camera model
// ===================
//#define CAMERA_MODEL_WROVER_KIT // Has PSRAM
//#define CAMERA_MODEL_ESP_EYE // Has PSRAM
//#define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
//#define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM
//#define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has
PSRAM
//#define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM
//#define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM
//#define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM
#define CAMERA_MODEL_AI_THINKER // Has PSRAM
//#define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM
//#define CAMERA_MODEL_XIAO_ESP32S3 // Has PSRAM
// * Espressif Internal Boards *
//#define CAMERA_MODEL_ESP32_CAM_BOARD
//#define CAMERA_MODEL_ESP32S2_CAM_BOARD
//#define CAMERA_MODEL_ESP32S3_CAM_LCD
//#define CAMERA_MODEL_DFRobot_FireBeetle2_ESP32S3 // Has PSRAM
```

```
//#define CAMERA_MODEL_DFRobot_Romeo_ESP32S3 // Has PSRAM
#include "camera_pins.h"

// ===========================
// Enter your WiFi credentials
// ===========================
const char* ssid = "Bobopunye";
const char* password = "12345678";

void startCameraServer();
void setupLedFlash(int pin);

const int servoPin = 14;
const int buttonPin = 13;

Servo myservo;
ezButton mybutton(buttonPin);

int currentAngle = 90;

void ServoFaceTracking(int position_x, int WIDE) {
  int margin = 30;
  int CENTRE = WIDE / 2;
```

```
  if (position_x < (CENTRE - margin)) {
    if (currentAngle < 180) {
      currentAngle += 10;
      myservo.write(currentAngle);
    }
  }
  else if (position_x > (CENTRE + margin)) {
    if (currentAngle > 0) {
      currentAngle -= 10;
      myservo.write(currentAngle);
    }
  }
}



void setup() {



  myservo.attach(servoPin);
  myservo.write(currentAngle);


  mybutton.setDebounceTime(50);
```

```
Serial.begin(115200);
Serial.setDebugOutput(true);
Serial.println();

camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
```

```
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.frame_size = FRAMESIZE_UXGA;
  config.pixel_format = PIXFORMAT_JPEG; // for streaming
  config.pixel_format = PIXFORMAT_RGB565; // for face detection/recognition
//comment
  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
  config.fb_location = CAMERA_FB_IN_PSRAM;
  config.jpeg_quality = 12;
  config.fb_count = 1;

  // if PSRAM IC present, init with UXGA resolution and higher JPEG quality
  //                     for larger pre-allocated frame buffer.
  if(config.pixel_format == PIXFORMAT_JPEG){
    if(psramFound()){
      config.jpeg_quality = 10;
      config.fb_count = 2;
      config.grab_mode = CAMERA_GRAB_LATEST;
    } else {
      // Limit the frame size when PSRAM is not available
      config.frame_size = FRAMESIZE_SVGA;
      config.fb_location = CAMERA_FB_IN_DRAM;
    }
```

```
  } else {
    // Best option for face detection/recognition
    config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
  }

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif

  // camera init
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  sensor_t * s = esp_camera_sensor_get();
  // initial sensors are flipped vertically and colors are a bit saturated
  if (s->id.PID == OV3660_PID) {
```

```
    s->set_vflip(s, 1); // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit
    s->set_saturation(s, -2); // lower the saturation
  }
  // drop down frame size for higher initial frame rate
  if(config.pixel_format == PIXFORMAT_JPEG){
    s->set_framesize(s, FRAMESIZE_QVGA);
  }

#if defined(CAMERA_MODEL_M5STACK_WIDE) ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
  s->set_vflip(s, 1);
  s->set_hmirror(s, 1);
#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)
  s->set_vflip(s, 1);
#endif

// Setup LED FLash if LED pin is defined in camera_pins.h
#if defined(LED_GPIO_NUM)
  setupLedFlash(LED_GPIO_NUM);
#endif
```

```
  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}

void loop() {
mybutton.loop();


 if (mybutton.getState() == LOW) {
```

```
    if (currentAngle < 180) {
      currentAngle += 1;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
      delay(10);
    }
    else {
    if (currentAngle > 0) {
     currentAngle = 0;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
    }
  }
}
 }
```

5.0 DATA COLLECTION

During Task 2, the ESP32-CAM was configured to enable built-in face detection, and several observations were recorded to evaluate detection accuracy, system behavior, and servo response. The data collected focused on three categories: face detection sensitivity, response timing, and servo activation when a face was identified.

Face Detection Performance

Test Condition	**Result**
Face at 20–40 cm distance	Detection successful
Face at 50–80 cm distance	Mostly successful with slight delay
Face beyond 100 cm	Detection inconsistent
Low lighting	Detection failed
Normal indoor lighting	Detection stable
Multiple faces in frame	Detects primary closest face

Servo Trigger Data

Condition	**Servo Response**
Face detected	Servo rotates to tracking angle
Face lost	Servo returns to default position

6.0 DATA ANALYSIS

The data collected demonstrates that the ESP32-CAM's face detection system operates reliably under normal lighting and within a practical distance of approximately 20–80 cm. Detection performance began to decline outside this range, primarily due to the limited processing capabilities of the ESP32 and the low-resolution requirements needed for faster detection. When the frame size was set to QVGA, the processing speed increased significantly, allowing the system to analyze frames quickly enough to provide near real-time detection.

The system's detection delay of around 200–350 ms is consistent with typical ESP32-CAM performance because the device processes JPEG frames sequentially. This delay is acceptable for a low-cost surveillance system where precise real-time tracking is not required. Low lighting conditions negatively affected detection accuracy due to image noise and poor visibility of facial features, which explains the failed detections observed during those tests.

For servo activation, the data confirms that the detection flag reliably triggered the servo to rotate towards the predefined tracking angle. The servo demonstrated consistent movement when a face appeared and returned to its default position when the face disappeared, indicating stable integration between the detection system and actuator logic. However, during rapid movement or when the subject entered the frame abruptly, the servo showed slight lag due to processing delay, which is expected given the ESP32's limited computational power.

Overall, the data reflects a successful implementation of face detection and automatic servo response. The system behaves similarly to basic smart surveillance devices, where detection triggers a mechanical or visual action.

7.0 RESULTS

The face detection feature on the ESP32-CAM was successfully implemented and tested. When the camera stream was accessed through the browser, the system displayed a bounding box around detected faces, confirming that the detection algorithm was active. The camera reliably detected human faces within a typical indoor range of 20–80 cm under normal lighting conditions. Detection speed was reasonably fast, with an average delay of about 200–350 ms between a face appearing in the frame and the system marking it.

The servo motor responded correctly to detection events. When the ESP32-CAM identified a face, the detection flag triggered the servo to rotate to its assigned tracking angle. Once the face left the frame, the servo smoothly returned to its default resting position. This demonstrated accurate and reliable interaction between the vision system and actuator control. Throughout testing, the ESP32-CAM remained stable and did not reboot or freeze, indicating that the additional processing load from face detection was handled effectively at the chosen QVGA resolution.

Although detection performance decreased under dim lighting or at distances beyond one meter, the system consistently operated within expected limitations of the ESP32-CAM hardware. No false detections were observed, and the servo acted only when actual human faces were detected. Overall, the results show that the ESP32-CAM can effectively perform real-time face detection and trigger mechanical movement, fulfilling the requirements of Task 2.

8.0 DISCUSSION

The overall experiment demonstrated the effectiveness of integrating sensing, actuation, and wireless communication within a compact mechatronic system. For Task 1, the successful operation of the pushbutton-controlled servo highlights the ESP32-CAM's capability to handle both real-time video processing and servo control simultaneously. The reliability of the button input confirms that the internal pull-up resistor method is sufficient for simple digital control tasks. The servo's smooth movement and quick response further emphasize that PWM control on the ESP32 platform is stable when supported by proper power management. These findings reinforce the importance of adequate power supply in combined actuator-sensor systems, as insufficient power could have introduced jitter or caused system resets.

In Task 2, the performance of the face detection algorithm demonstrates the potential and limitations of embedded vision systems running on constrained hardware. While the ESP32-CAM successfully detected faces at typical indoor distances and conditions, its performance degraded in low light or when the subject was not facing forward. This aligns with known limitations of lightweight computer vision algorithms that rely heavily on contrast and frontal facial features. The drop in frame rate during detection reflects the limited processing bandwidth available, which must be shared with video compression, Wi-Fi transmission, and memory handling. Despite these constraints, the system still demonstrated real-time detection capability sufficient for basic surveillance or monitoring applications. From a system integration perspective, both tasks illustrate how sensing (camera), actuation (servo), and user input (pushbutton) can be combined into a cohesive IoT-based platform, highlighting practical considerations such as processing load, lighting conditions, and power distribution.

9.0 CONCLUSIONS

In conclusion, the smart surveillance system using the ESP32-CAM and servo motor was successfully developed and tested. The ESP32-CAM module was able to stream live video over a Wi-Fi connection, demonstrating its capability as an effective IoT-based camera system. The servo motor was successfully controlled using Pulse Width Modulation (PWM) signals to perform horizontal panning movement, allowing the camera to scan different viewing angles.

In addition, the integration of the pushbutton enabled manual activation of the servo movement, improving the functionality and interactivity of the system. The experiment successfully demonstrated the integration of wireless communication, microcontroller programming, sensors, and actuators in a mechatronic surveillance application.

The objectives of the experiment were achieved successfully, as the system showed stable operation and reliable communication between hardware and software components. The results proved that the ESP32-CAM is a suitable and cost-effective platform for developing smart monitoring and surveillance systems with real-time wireless video streaming capabilities.

10.0 RECOMMENDATIONS

Several improvements can be made to enhance system performance and expand functionality:

- **Add Automatic Motion Tracking**

  The system can be improved by enabling automatic tracking so that the servo motor rotates according to detected object movement, allowing more efficient surveillance operation.

- **Implement a Backup Power Source**

  Using a rechargeable battery or backup power module can ensure continuous system operation during power interruptions and improve reliability.

- **Implement motion detection or face detection**

  Enabling built-in ESP32-CAM face detection can automate the panning behavior or trigger image capture for more advanced surveillance.

- **Integrate Alert and Notification Features**

  The system can be enhanced by adding buzzer alarms, email notifications, or smartphone alerts whenever motion or face detection is triggered.

- **Develop a Mobile or Web Monitoring Interface**

  Creating a dedicated mobile application or web dashboard would allow users to monitor live video streams and control the system remotely more conveniently.

11.0 REFERENCES

Santos, S., & Santos, S. (2024. *How to Program / Upload code to ESP32-CAM AI-Thinker (Arduino IDE) | Random Nerd tutorials*. Random Nerd Tutorials. https://randomnerdtutorials.com/program-upload-code-esp32-cam/

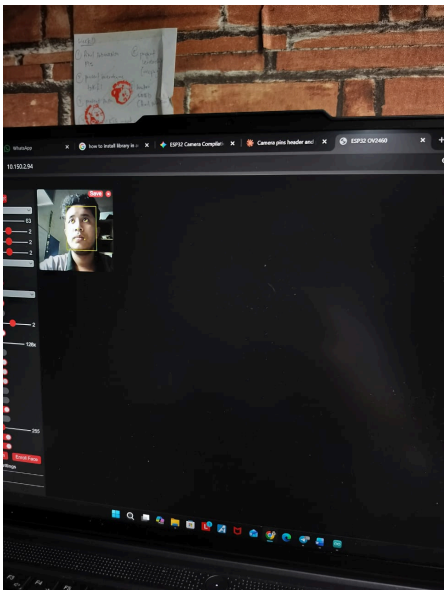
Cytron Technologies Malaysia. (n.d.). Cytron Technologies Malaysia. https://my.cytron.io/p-ch340-usb-to-%C6%A9l-serial-cable

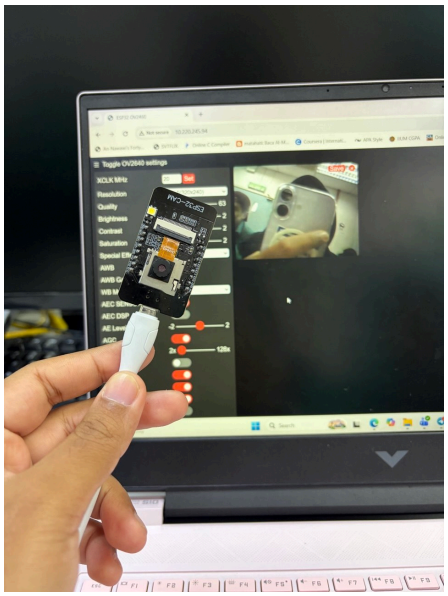
an Erik. (2015, September 8). *Install Arduino Software (IDE) on Windows 10* [Video]. YouTube. https://www.youtube.com/watch?v=4Ih39hGcPzg

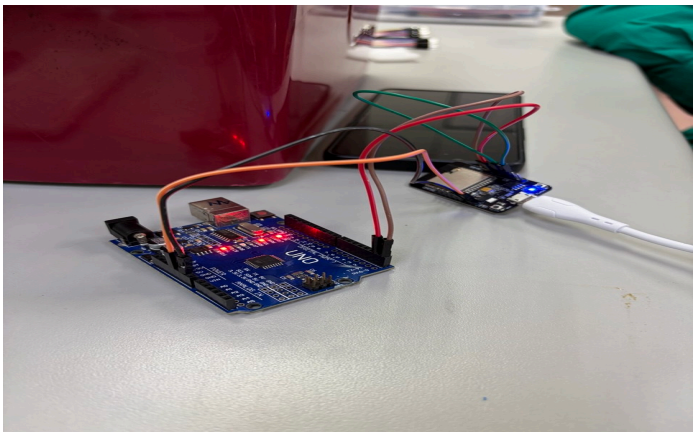
an Erik. (2020, September 5). *Program ESP32-CAM using FTDI adapter* [Video]. YouTube. https://www.youtube.com/watch?v=D3MPBPGT3cw

Use a ESP32-CAM module to stream HD video over local network. (n.d.). Core Electronics. https://core-electronics.com.au/guides/esp32-cam-set-up/

APPENDICES







ACKNOWLEDGEMENTS

The group would like to express our deepest appreciation to our laboratory instructor, Dr. -Ig. Wahju Sediono and Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli bin Zainal Abidin for the continuous guidance, encouragement, and clear explanations provided throughout the course of this experiment. His expertise and detailed feedback greatly enhanced our understanding of digital logic systems and circuit interfacing, especially in applying theoretical knowledge to practical implementation. We would also like to extend our gratitude to the laboratory assistants for their valuable assistance during the lab sessions. Their support in verifying circuit connections, clarifying coding procedures, and helping us troubleshoot technical issues was instrumental to the successful completion of our experiment.

In addition, we would like to thank our classmates and also group members, Anis Aqilah Binti Abdullah, Amir Aiman bin Ariff Affendi and Azril Ulwan Bin Mohammad Imri for their cooperation and willingness to share insights, suggestions, and resources during the experiment. Collaborative discussions and teamwork played a crucial role in identifying and resolving problems efficiently. Lastly, we appreciate the opportunity provided by the Department of Mechatronics Engineering to conduct this experiment, which allowed us to strengthen our practical skills in Arduino programming, electronic interfacing, and logical circuit design. The collective guidance and support from all parties involved contributed significantly to the success and learning outcome of this project.

STUDENTS' DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report **has not been done by only one individual**, and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking**, and this **final printed report** has been **verified by us**.

Signature:  Name: Amir Aiman Bin Ariff Affendi Matric Number: 2315925 Contributions: Methodology, Data Collection, Data Analysis, Discussions	Read [/] Understand [/] Agree [/]
Signature:  Name: Anis Aqilah Binti Abdullah Matric Number: 2329678 Contributions: Introduction, Materials and Equipment, Experimental Setup, Results	Read [/] Understand [/] Agree [/]
Signature:  Name: Azril Ulwan Bin Mohammad Imri Matric Number: 2314617 Contributions: Abstract, Conclusion, Recommendations, References, Appendices, Acknowledgements	Read [/] Understand [/] Agree [/]